

M1 Algebraic Algorithms for Cryptology: homework

This homework is to be done in groups of 1 or 2. Your answers should be written in the form of a pdf file, together with the several file of source code for the program requested. You should send all these files in a compressed archive (either in tar.gz or zip format) attached to an email to `clement.pernet@univ-grenoble-alpes.fr` with subject [M1 AAC] Homework <LAST NAME 1> < LAST NAME 2> by **Monday March 23th, 2026**. All answers must be justified.

This homework is your personnal work: any **use of LLM assistance is forbidden**. I will test your work for similarities : any **plagiarism and copy of someone else's work will be sanctioned**.

Exercice 1. Implementation of $\mathbb{Z}/n\mathbb{Z}$ and RSA

An implementation of a finite ring $\mathbb{Z}/n\mathbb{Z}$ consists in

- defining a type `Element` to represent its elements
- an addition function: `Element add (Element x, Element y);`
- a multiplication function: `Element mul (Element x, Element y);`
- a negation function: `Element neg (Element x);`
- a test for invertibility: `isInvertible (Element x);`
- an invert function: `invert (Element x);` which handles the case when x is not invertible

a. First, we will focus on an implementation of $\mathbb{Z}/n\mathbb{Z}$ using the type `uint64_t` for its elements (in C, this is achieved by the command `typedef uint64_t Element;`). What is the maximal value for n can this implementation handle ?

b. Implement all the functions described above. The modulus n can be either stored as a global variable, or as an additional parameter to each function.

c. Implement a modular exponentiation function, `modexp (Element x, uint64_t k)` computing $x^k \bmod n$.

d. In a separate file `test-znz.c`, write an executable program (with a main in C) that demonstrates the usage of these functions.

Recall that textbook RSA is defined by the following functionalities:

- $K_{\text{pub}} = (n, e)$, $K_{\text{priv}} = d$ where $n = pq$; p, q are prime numbers and $d = e^{-1} \bmod \varphi(n)$.
- $\text{Enc}(m, e) = m^e \bmod n$
- $\text{Dec}(c, d) = c^d \bmod n$

e. Propose an implementation of the **Enc**, **Dec** functions and a **Keygen** function, which takes as input the 2 prime numbers p and q .

f. In a separate file, **rsa-demo.c** write an executable program (with a main in C) that demonstrates the usage of these functions. You can use for this purpose the following prime numbers: 313486403, 521502889, 804582047, 529940513, 184789603, 981009509, 618567539, 117004469, 773296369, 2402710

g. (Optional) Propose an implementation of multi-precision integers

h. (Optional) Redo as many as possible of this exercise using your multi-precision implementation to handle finite ring $\mathbb{Z}/n\mathbb{Z}$ of arbitrary size.

Exercice 2. Implementation of binary Galois fields $\mathbb{F}_{128}$

a. Recall how is the field $\mathbb{F}_{128}$ defined. In particular, explain

1. the representation of its elements;
2. the principle of the field addition;
3. the principle of the field multiplication;
4. the principle of the field inversion.

b. Among the following polynomials, which one can you use to construct this field:

1. $P_1 = X^{128} + X + 1$
2. $P_3 = X^7 + X + 1$
3. $P_4 = X^7 + X^4 + X + 1$
4. $P_2 = X^8 + X + 1$
5. $P_5 = X^8 + X^4 + X^3 + X + 1$

c. Implement all the field operations in C in a file **f128.c**: addition, negation, subtraction, multiplication, inversion, division.

d. Write an executable in a file **f128-demo.c** demonstrating the usage of your code. We now focus on a Zech-log representation (see lecture slides)

e. Explain the principle of this representation.

f. What additional condition should satisfy the polynomial used for the construction of the field? Prove that all polynomials used for constructing $\mathbb{F}_{128}$ satisfy this additional condition.

g. Propose another implementation of $\mathbb{F}_{128}$ based on Zech logarithm implementation in a file **f128-zech.c**.

h. Write an executable in a file **f128-zech-demo.c** demonstrating the usage of your code.